

```
--- backup\prueba\insert_build.py ---
```

```
#scripts/con_psycomp/builds
```

```
import psycomp
```

```
from psycomp.rows import dict_row
```

```
# Parámetros de conexión de la Práctica 1
```

```
DB_NAME = "exam"
```

```
DB_USER = "postgres"
```

```
DB_PASS = "postgres"
```

```
DB_HOST = "postgis"
```

```
DB_PORT = "5432"
```

```
# Parámetros p1Settings
```

```
EPSG_CODE = 25830
```

```
TOLERANCE = 0.0001
```

```
def connect():
```

```
    """Establece la conexión a la base de datos devolviendo diccionarios."""
```

```
    conn = psycomp.connect(
```

```
        dbname=DB_NAME,
```

```
        user=DB_USER,
```

```
        password=DB_PASS,
```

```
        host=DB_HOST,
```

```
        port=DB_PORT,
```

```
        row_factory=dict_row # Devuelve filas nativas como diccionarios
```

```
    )
```

```
    return conn
```

```
# FUNCIONES PREVIAS DE VALIDACIÓN
```

```
def snap_geometry(conn, geom_wkt):
```

```
    """
```

```
    Aplica ST_SnapToGrid y devuelve el texto de la geometría procesada.
```

```
    """
```

```
    with conn.cursor() as cur:
```

```
        sql = "SELECT ST_AsText(ST_SnapToGrid(ST_GeomFromText(%s, %s), %s)) AS
```

```
geom_procesada;"
```

```
        cur.execute(sql, (geom_wkt, EPSG_CODE, TOLERANCE))
```

```
        resultado = cur.fetchone()
```

```
        return resultado['geom_procesada'] if resultado else None
```

```
def check_st_intersects(conn, geom_wkt):
```

```
    """Control de datos de entrada: ST_Intersects."""
```

```
    with conn.cursor() as cur:
```

```
        sql = "SELECT id FROM d.buildings WHERE ST_Intersects(geom, ST_GeomFromText(%s, %s));"
```

```
        cur.execute(sql, (geom_wkt, EPSG_CODE))
```

```
        return cur.fetchone()
```

```
def check_st_within(conn, geom_wkt):
```

```
    """Control de datos de entrada: ST_Within."""
```

```
    with conn.cursor() as cur:
```

```

        sql = "SELECT id FROM d.buildings WHERE ST_Within(ST_GeomFromText(%s, %s),
geom);"
        cur.execute(sql, (geom_wkt, EPSG_CODE))
        return cur.fetchone()

def check_st_relate(conn, geom_wkt, matrix='T*****'):
    """
    Control topológico usando 9IM Matrix.
    Por defecto 'T*****' busca intersección estricta de interiores.
    """
    with conn.cursor() as cur:
        sql = "SELECT id FROM d.buildings WHERE ST_Relate(geom, ST_GeomFromText(%s, %s),
%s);"
        cur.execute(sql, (geom_wkt, EPSG_CODE, matrix))
        return cur.fetchone()

# INSERT
def insert(d: dict):
    """
    Recibe un diccionario, utiliza las funciones previas para validar y procesar,
    y retorna el resultado estandarizado requerido por el examen.
    """
    # 1. Extraer datos del diccionario
    geom_wkt = d.get('geom_wkt')
    description = d.get('description', '')
    area = d.get('area', 0.0)

    if not geom_wkt:
        return {'ok': False, 'message': 'No se proporcionó geom_wkt', 'data': []}

    conn = None
    try:
        conn = connect()

        # 2. Procesar la geometría con la función previa de Snapping
        snapped_wkt = snap_geometry(conn, geom_wkt)

        if not snapped_wkt:
            return {'ok': False, 'message': 'Fallo al procesar la geometría', 'data':
[]}]

        # 3. Validar con la función previa (usamos relate según recomendación del
manual)
        conflict = check_st_relate(conn, snapped_wkt)
        # conflict = check_st_intersects(conn, snapped_wkt)
        # conflict = check_st_within(conn, snapped_wkt)

        if conflict:
            # Como usamos dict_row, 'conflict' es un dict: {'id': 15}
            return {
                'ok': False,
                'message': f"Error topológico: Intersecta con el edificio ID
{conflict['id']}",

```

```

        'data': []
    }

# 4. Inserción de los datos usando la geometría ya procesada
with conn.cursor() as cur:
    insert_sql = """
        INSERT INTO d.buildings (description, area, geom)
        VALUES (%s, %s, ST_GeomFromText(%s, %s))
        RETURNING id, description, area, ST_AsText(geom) as geom_wkt;
    """
    cur.execute(insert_sql, (description, area, snapped_wkt, EPSG_CODE))
    new_row = cur.fetchone() # Capturamos todo el nuevo registro en diccionario

    conn.commit()

    return {
        'ok': True,
        'message': 'Building inserted',
        'data': [new_row]
    }

except Exception as e:
    if conn:
        conn.rollback()
    return {'ok': False, 'message': f'Error SQL: {str(e)}', 'data': []}

finally:
    # Cierre OBLIGATORIO de conexiones
    if conn:
        conn.close()

if __name__ == "__main__":
    # Diccionario simulando la entrada de prueba
    test_dict = {
        'description': 'Almacén Validado Modular',
        'area': 1500.50,
        'geom_wkt': 'POLYGON ((711624.4 4245616.4, 711750.3 4245550.7, 711735.1
4245525.5, 711612.2 4245591.7, 711624.4 4245616.4))'
    }

    # Ejecutamos la función de inserción
    resultado = insert(test_dict)
    print(resultado)

# cd scripts/con_psycomp/builds
# python3 insert_builds.py

```